Git hub:https://github.com/hemanth1364/TCS-questions

Question 1:

Problem Name: Chocolate Factory

Problem Statement –

A chocolate factory is packing chocolates into the packets. The chocolate packets here represent an
array of N number of integer values. The task is to find the empty packets(0) of chocolate and push it
to the end of the conveyor belt(array).

Example 1 :

N=8 and arr = [4,5,0,1,9,0,5,0].

There are 3 empty packets in the given set. These 3 empty packets represented as O should be pushed
towards the end of the array

Input :

8 – Value of N

[4,5,0,1,9,0,5,0] – Element of arr[O] to arr[N-1],While input each element is separated by newline

Output:

4 5 1 9 5 0 0 0

Example 2:

Input:

6 — Value of N.

[6,0,1,8,0,2] – Element of arr[0] to arr[N-1], While input each element is separated by newline

Output:

6 1 8 2 0

Code:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int j = 0;
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        for (int i = 0; i < n; i++)
            if (arr[i] != 0)
                arr[j++] = arr[i];
        while (j < n)
            arr[j++] = 0;
        for (int x : arr)
            System.out.print(x + " ");
    }
}
```

OUTPUT:
4 5 1 9 5 0 0 0

Question 2 :
Problem Name: Toggling Bits
Problem Statement –
Joseph is learning digital logic subject which will be for his next semester. He usually tries to solve
unit assignment problems before the lecture. Today he got one tricky question. The problem statement
is “A positive integer has been given as an input. Convert decimal value to binary representation.
Toggle all bits of it after the most significant bit including the most significant bit. Print the positive
integer value after toggling all bits”.
Constraints 1<=N<=100
Example 1:
Input :
10 -> Integer

Output :
5 -> result- Integer
Explanation:
Binary representation of 10 is 1010. After toggling the bits(1010), will get 0101 which represents “5”.
Hence output will print “5”.

CODE :

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int bits = Integer.toBinaryString(n).length();
        int mask = (1 << bits) - 1;

        System.out.println(n ^ mask);
    }
}
```

Input:
10

Binary Representation:

10 = 1010

After Toggling All Bits:

1010 → 0101

OUTPUT:

5

Question 3 :

Problem Name: Count Sundays

Problem Statement –

Jack is always excited about sunday. It is favourite day, when he gets to play all day. And goes to
cycling with his friends.

So every time when the months starts he counts the number of sundays he will get to enjoy.

Considering the month can start with any day, be it Sunday, Monday…. Or so on.

Count the number of Sunday jack will get within n number of days.

Example 1:

Input

mon-> input String denoting the start of the month.

13 -> input integer denoting the number of days from the start of the month.

Output :

2 -> number of days within 13 days.

Explanation:

The month start with mon(Monday). So the upcoming sunday will arrive in next 6 days. And then
next Sunday in next 7 days and so on.

Now total number of days are 13. It means 6 days to first sunday and then remaining 7 days will end
up in another sunday. Total 2 sundays may fall within 13 days.

CODE :

```
import java.util.*;

public class Main {

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    String startDay = sc.next();

    int date = sc.nextInt();
```

```
        String[] days = {"sun", "mon", "tue", "wed", "thu", "fri", "sat"};
        int start = 0;
        for (int i = 0; i < 7; i++) {
            if (days[i].equals(startDay)) {
                start = i;
                break;
            }
        }
        int result = (start + date - 1) % 7;
        System.out.println(days[result]);
    }
}
```

INPUT :

Mon 13

OUTPUT:

sat

Formula Used: (startDayIndex + date - 1) % 7

TCS – Set 2 Questions

Question 4 :

Problem Name: Risk Severity

Problem Statement –

Airport security officials have confiscated several item of the passengers at the security check point.

All the items have been dumped into a huge box (array). Each item possesses a certain amount of

risk[0,1,2]. Here, the risk severity of the items represent an array[] of N number of integer values. The

task here is to sort the items based on their levels of risk in the array. The risk values range from 0 to

2.

Example :

Input :

7 -> Value of N

[1,0,2,0,1,0,2]-> Element of arr[0] to arr[N-1], while input each element is separated by new line.

Output :

0 0 0 1 1 2 2 -> Element after sorting based on risk severity

Example 2:

input : 10 -> Value of N

[2,1,0,2,1,0,0,1,2,0] -> Element of arr[0] to arr[N-1], while input each element is separated by a new

line.

Output :

0 0 0 0 1 1 1 2 2 2 ->Elements after sorting based on risk severity.

Explanation:

In the above example, the input is an array of size N consisting of only 0’s, 1’s and 2s. The output is a

sorted array from 0 to 2 based on risk severity.

CODE:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int c0 = 0, c1 = 0, c2 = 0;

        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            if (x == 0) c0++;
            else if (x == 1) c1++;
            else c2++;
        }
        while (c0-- > 0) System.out.print("0 ");
        while (c1-- > 0) System.out.print("1 ");
        while (c2-- > 0) System.out.print("2 ");
    }
}
```

Input:
10
2
1
0
2
1
0
0
1
2
0

Output:

0 0 0 0 1 1 1 2 2 2

Question 5 :
Problem Name: Prior Elements
Problem Statement –
Given an integer array Arr of size N the task is to find the count of elements whose value is greater
than all of its prior elements.
Note : 1st element of the array should be considered in the count of the result.
For example,
Arr[]={7,4,8,2,9}
As 7 is the first element, it will consider in the result.
8 and 9 are also the elements that are greater than all of its previous elements.
Since total of 3 elements is present in the array that meets the condition.
Hence the output = 3.
Example 1:
Input
5 -> Value of N, represents size of Arr
7-> Value of Arr[0]
4 -> Value of Arr[1]
8-> Value of Arr[2]
2-> Value of Arr[3]
9-> Value of Arr[4]
Output :
3
Example 2:
5 -> Value of N, represents size of Arr
3 -> Value of Arr[0]
4 -> Value of Arr[1]
5 -> Value of Arr[2]
8 -> Value of Arr[3]
9 -> Value of Arr[4]
Output :
5
Constraints
1<=N<=20
1<=E

CODE:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int max = 0, count = 0;

        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();

            if (i == 0 || x > max) {
                count++;
                max = x;
            }
        }

        System.out.println(count);
    }
}
```

INPUT:

```
5
7
4
8
2
9
```

OUTPUT:

```
3
```

Shortcut to Remember:

Count an element if it is **greater than all elements before it**. Keep updating the maximum value seen so far.

Question 6 :

Problem Name: Pricing Format

Problem Statement –

A supermarket maintains a pricing format for all its products. A value N is printed on each product.

When the scanner reads the value N on the item, the product of all the digits in the value N is the price

of the item. The task here is to design the software such that given the code of any item N the product

(multiplication) of all the digits of value should be computed(price).

Example 1:

Input :

5244 -> Value of N

Output :

160 -> Price

Explanation:

From the input above

Product of the digits 5,2,4,4

5*2*4*4= 160

Hence, output is 160.

CODE:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
```

```
        int product = 1;

        while (n > 0) {
            product *= n % 10;
            n /= 10;
        }

        System.out.println(product);
    }
}
```

Input:
5244
Output:
160

Question 7 :

TCS – Set 3 Questions

Problem Name: Collection of Curtains
Problem Statement –
A furnishing company is manufacturing a new collection of curtains. The curtains are of two colors
aqua(a) and black (b). The curtains color is represented as a string(str) consisting of a's and b's of
length N. Then, they are packed (substring) into L number of curtains in each box. The box with the
maximum number of 'aqua' (a) color curtains is labeled. The task here is to find the number of 'aqua'
color curtains in the labeled box.
Note :
If 'L' is not a multiple of N, the remaining number of curtains should be considered as a substring too.

In simple words, after dividing the curtains in sets of ‘L’, any curtains left will be another set(refer
example 1)
Example 1:
Input :
bbbaaababa -> Value of str
3 -> Value of L
Output:
3 -> Maximum number of a’s
Explanation:
From the input given above.
Dividing the string into sets of 3 characters each
Set 1: {b,b,b}
Set 2: {a,a,a}
Set 3: {b,a,b}
Set 4: {a} -> leftover characters also as taken as another set
Among all the sets, Set 2 has more number of a’s. The number of a’s in set 2 is 3.
Hence, the output is 3.
Example 2:
Input :
abbbaabbb -> Value of str

CODE :

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int L = sc.nextInt();

        int max = 0;

        for (int i = 0; i < str.length(); i += L) {
            int count = 0;
```

```
            for (int j = i; j < Math.min(i + L, str.length()); j++) {
                if (str.charAt(j) == 'a')
                    count++;
            }

            max = Math.max(max, count);
        }

        System.out.println(max);
    }
}
```

Input:
abbbaabbb
3
Output :
2

Question 8 :
Problem Name: Table Conference
Problem Statement –
An international round table conference will be held in india. Presidents from all over the world
representing their respective countries will be attending the conference. The task is to find the possible
number of ways(P) to make the N members sit around the circular table such that.
The president and prime minister of India will always sit next to each other.
Example 1:
Input :
4 -> Value of N(No. of members)
Output :
12 -> Possible ways of seating the members
Explanation:

2 members should always be next to each other.
So, 2 members can be in 2!ways
Rest of the members can be arranged in (4-1)! ways.(1 is subtracted because the previously selected
two members will be considered as single members now).
So total possible ways 4 members can be seated around the circular table 2*6= 12.
Hence, output is 12.
Example 2:
Input:
10 -> Value of N(No. of members)
Output :
725760 -> Possible ways of seating the members
Explanation:
2 members should always be next to each other.
So, 2 members can be in 2! ways
Rest of the members can be arranged in (10-1)! Ways. (1 is subtracted because the previously selected
two members will be considered as a single member now).
So, total possible ways 10 members can be seated around a round table is
2*362880 = 725760 ways.
Hence, output is 725760.
The input format for testing
The candidate has to write the code to accept one input
First input – Accept value of number of N(Positive integer number)
The output format for testing
The output should be a positive integer number or print the message(if any) given in the problem
statement(Check the output in example 1, example2)
Constraints :
2<=N<=50

CODE:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        long fact = 1;
        for (int i = 2; i <= n - 2; i++) {
            fact *= i;
        }

        System.out.println(2 * fact);
    }
}
```

Input:
10
Output
80640

Question 9 :
Problem Name: Mysterious Method
Problem Statement –
An intelligence agency has received reports about some threats. The reports consist of numbers in a
mysterious method. There is a number “N” and another number “R”. Those numbers are studied
thoroughly and it is concluded that all digits of the number ‘N’ are summed up and this action is
performed ‘R’ number of times. The resultant is also a single digit that is yet to be deciphered. The
task here is to find the single-digit sum of the given number ‘N’ by repeating the action ‘R’ number of
times.
If the value of ‘R’ is 0, print the output as ‘0’.
Example 1:
Input :
99 -> Value of N
3 -> Value of R

Output :
9 -> Possible ways to fill the cistern.
Explanation:
Here, the number N=99
1. Sum of the digits N: 9+9 = 18
2. Repeat step 2 ‘R’ times i.e. 3 tims (9+9)+(9+9)+(9+9) = 18+18+18 =54
3. Add digits of 54 as we need a single digit 5+4
Hence , the output is 9.
Example 2:
Input :
1234 -> Value of N
2 -> Value of R
Output :
2 -> Possible ways to fill the cistern
Explanation:
Here, the number N=1234
1. Sum of the digits of N: 1+2+3+4 =10
2. Repeat step 2 ‘R’ times i.e. 2 times (1+2+3+4)+(1+2+3+4)= 10+10=20
3. Add digits of 20 as we need a single digit. 2+0=2
Hence, the output is 2.
Constraints:
0<N<=1000
0<=R<=50
The Input format for testing
The candidate has to write the code to accept 2 input(s)
First input- Accept value for N (positive integer number)
Second input: Accept value for R(Positive integer number)
The output format for testing
The output should be a positive integer number or print the message (if any) given in the problem
statement. (Check the output in Example 1, Example 2).

CODE:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
        int r = sc.nextInt();

        if (r == 0) {
            System.out.println(0);
            return;
        }
        int sum = 0;

        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        sum *= r;

        while (sum > 9) {
            int temp = 0;

            while (sum > 0) {
                temp += sum % 10;
                sum /= 10;
            }

            sum = temp;
        }

        System.out.println(sum);
    }
}
```

INPUT:
1234
2

Output :
2

Easy Logic
1. Sum digits of N.
2. Multiply by R.
3. Keep summing digits until a single digit remains.
4. If R = 0, print 0.